Structuring WhatsApp Chats with Multimodal AI: Expense Parsing and Classification with Chetto.ai

Internship Project Report

By

Anannya Chaudhary

52

12111468

Under the guidance of

Mr Somasunder S | **Prof Surabhi Kakade**

SecureText Research Labs | **VIT**

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

BRACT'S VISHWAKARMA INSTITUTE TECHNOLOGY

PUNE – 411 037

BANSILAL RAMNATH AGARWAL CHARITABLE TRUST'S

VISHWAKARMA INSTITUTE TECHNOLOGY

PUNE – 411 037

CERTIFICATE

Certified that this Project report titled "Structuring WhatsApp Chats with Multimodal AI: Expense Parsing and Classification with Chetto.ai" by "Anannya Chaudhary (52)" is approved by me for submission. Certified further that, to the best of my knowledge, the report represents work carried out by the student as the project as prescribed by the Vishwakarma Institute of Technology Pune in the academic year 2024-25.

Date: 9th May 2025 **Guide:** Prof Surabhi Kakade

Head of Department: Prof Shital Dongre

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to those who supported and guided me throughout the completion of this project.

I am thankful to my project guide, Mr. Somasunder S from SecureText Research Labs, for his mentorship, valuable insights, and encouragement during the course of my internship. His insights were instrumental in shaping the multimodal AI pipeline for structuring unstructured WhatsApp conversations—particularly in the design and refinement of flow-based systems for expense classification, document captioning, and intelligent message parsing in Chetto.ai.

I also extend my appreciation to Prof. Surabhi Kakade of Vishwakarma Institute of Technology (VIT) for her academic guidance, constructive feedback, and support. I am grateful to the Department of Artificial Intelligence and Data Science at Vishwakarma Institute of Technology, Pune, for providing me with this opportunity to work on this project.

Anannya Chaudhary

CONTENTS

1. Abstract

Managing and extracting structured insights from informal WhatsApp communication presents unique challenges, particularly when it involves varied content formats such as text, images, and PDFs. This internship focused on the development and refinement of Chetto.ai—an AI-driven platform that automates the classification and extraction of expense-related data from WhatsApp group chats using a multimodal approach. The work involved improving the OCR pipeline for better text extraction, fine-tuning BERT-based models for contextual classification, and integrating Florence-2-large to generate captions from documents and images. These enhancements enabled reliable differentiation between expense and non-expense content across diverse inputs. Additionally, key bugs were resolved in Prohirify's resume parsing system, improving its accuracy and stability. This project demonstrates how AI can streamline unstructured communication channels into actionable workflows through intelligent automation.

2. Introduction

WhatsApp has become the backbone of communication for many Indian businesses, especially in the informal and small-scale sector. While it offers convenience, it also creates a flood of unstructured data—text, images, documents, and voice notes—that are difficult to monitor, analyze, or convert into action. Chetto.ai emerged as a solution to this growing chaos, using multimodal AI to bring structure, clarity, and intelligence to WhatsApp conversations. This project contributes to Chetto's mission by enhancing its ability to parse, classify, and manage expense-related information and financial projections within chat-based business workflows.

2.1 Importance of WhatsApp-Based Communication in Indian Businesses

In India, WhatsApp is not just a messaging app—it is a business tool. Countless micro, small, and medium enterprises (MSMEs), as well as startups and informal teams, rely on WhatsApp for their daily operations, sharing everything from invoices to client tasks and projections. However, the informal and unstructured nature of these conversations makes it difficult to organize, track, and analyze essential business information.

Chetto.ai addresses this challenge by transforming chaotic WhatsApp threads into structured, searchable, and actionable data. By applying advanced multimodal AI, the platform helps teams improve efficiency, financial accountability, and overall decision-making—all without disrupting the natural flow of their communication.

2.2 Challenges in Organizing WhatsApp Chat Data

Unstructured data from WhatsApp poses significant technical and operational challenges. Businesses often miss critical expense updates or end up duplicating tasks due to the overwhelming volume of messages, forwarded content, and multimedia data.

Additionally, there is little to no native support within WhatsApp for tracking tasks, documenting expenses, or distinguishing between actionable and non-actionable content. Manual monitoring is time-consuming, error-prone, and unsustainable at scale.

2.2.1 Multimodal Nature of Data and Redundancy in Communication

WhatsApp chats include a mix of text, images, PDFs, screenshots, and even emojis or voice notes. This diversity introduces several complexities:

- OCR systems must be robust enough to extract text from low-quality or cluttered media.
- AI models need to understand both visual and textual context to classify whether a message is expense-related.
- Frequent repetition of projections can lead to task duplication unless models can detect semantic similarity across messages.

2.2.2 Objectives of the Project

The key objectives of this internship were to contribute to the intelligent parsing and classification systems in Chetto by:

- Improving the OCR pipeline to enhance text extraction from varied expense documents such as invoices and bills.
- Fine-tuning a BERT-based model to detect similarity in projections to prevent redundant task creation in the system and classify expenses from all the PDF’s and images.
- Integrating Florence-2-large in the pipeline for captioning PDFs and images to support multimodal classification of WhatsApp data into expense and non-expense categories.
- Debugging and optimizing resume parsing in the Prohirify platform by identifying and resolving critical issues that impacted extraction accuracy.

3. Previous Work / Existing System Study

3.1 Existing Systems, Algorithms, and Methods

In the realm of business communication, especially within Indian SMEs and informal sectors, WhatsApp has become a primary tool for daily operations. Recognizing this, several tools have emerged aiming to structure and automate WhatsApp interactions. Notable among these are:

- Gallabox: Offers no-code WhatsApp chatbots tailored for lead generation and customer support. While effective for predefined workflows, it lacks advanced AI capabilities for nuanced data extraction from diverse message types.
- Landbot: Provides a visual interface for chatbot creation, focusing on customer engagement. However, its strength lies in structured interactions rather than parsing unstructured data like images or documents.
- Infobip's WhatsApp Business API: Facilitates automated messaging and notifications. While robust in message delivery, it doesn't inherently offer AI-driven data classification or extraction from multimedia content.
- YCloud: An AI-focused WhatsApp API platform emphasizing marketing automation. Though it integrates AI, its primary focus is on marketing campaigns rather than comprehensive chat data structuring.

These tools primarily address customer engagement and support, offering automation for repetitive tasks. However, they often lack the depth required for comprehensive data extraction and classification from the multifaceted nature of WhatsApp communications, especially in contexts where messages include a mix of text, images, and documents.

3.2 Drawbacks of Existing Systems

Despite the advancements these tools bring, several limitations persist:

- Limited Multimodal Data Handling: Most tools are optimized for text-based interactions. They struggle with extracting and interpreting data from images, PDFs, or other non-textual content commonly shared in business communications.
- Inadequate Contextual Understanding: The absence of advanced AI models means these systems often fail to grasp the context of messages, leading to misclassification or oversight of critical information.
- Lack of Semantic Similarity Detection: In environments where projections or reports are frequently updated and shared, existing tools don't effectively identify and manage duplicate or similar content, leading to clutter and inefficiencies.
- Insufficient Localization: Many tools don't cater to the linguistic diversity present in Indian businesses, limiting their accessibility and effectiveness across different regions.

- Data Privacy Concerns: Tools like WhatsApp Business App have faced scrutiny over data protection and privacy compliance, especially concerning metadata handling.

3.3 Problem Definition

This project aims to bridge the gaps identified in existing systems by developing an AI-driven solution tailored for structuring WhatsApp communications in Indian business contexts. The primary challenges addressed include:

- Multimodal Data Parsing: Implementing advanced OCR and AI models to extract and classify information from various content types, including images, PDFs, and text messages.
- Contextual and Semantic Analysis: Utilizing fine-tuned models like BERT to understand the context of messages, enabling accurate classification and duplication detection.
- Localization and Accessibility: Ensuring the system supports multiple Indian languages, making it accessible to a broader range of users.
- Enhanced Data Privacy: Designing the system with robust data protection measures to address privacy concerns inherent in existing tools.

By focusing on these areas, the project endeavors to create a comprehensive solution that transforms unstructured WhatsApp communications into structured, actionable data, thereby enhancing efficiency and decision-making in Indian businesses.

3.4 Requirement Specification

The Chetto platform addresses the need to intelligently organize WhatsApp business chats by extracting expenses, classifying messages, and detecting projection similarities. The following are the key functional and non-functional requirements identified through system design and flowchart analysis.

3.4.1 Functional Requirements

WhatsApp Message Parsing and Preprocessing:

- The system must extract raw messages from WhatsApp chats (via APIs or exported chat logs).
- It should segment messages by sender and time, and tag system notifications or media attachments appropriately.

Expense Parsing and Extraction:

- The system should detect expense-related messages and extract structured financial data (e.g., item names, amounts, dates).
- OCR pipelines should process image-based receipts using OCR tools like (e.g., Florence-2-large, Pytesseract OCR) to retrieve expenses.

- Parsed expenses must be normalized and linked to conversation context (e.g., who paid, what for, when).

Projection Similarity Detection using Fine-Tuned BERT:

- The system should use a fine-tuned transformer (BERT) to compare messages containing projections (e.g., sales targets or estimates).
- It must classify whether two projections are similar or distinct to prevent redundancy in the user interface.
- Similar projections should be grouped to reduce noise, while distinct ones should be prominently displayed.

Message Classification using Multimodal Models:

- The system should leverage a model like Florence-2-large to classify WhatsApp messages (pdfs and images) into business-relevant categories (e.g., sales, expenses etc).
- The classification system must be capable of processing both text-only and text-with-image inputs.

User Interface Integration (via Chetto.ai):

- Processed information should be integrated into the Chetto dashboard for users to view categorized messages, extracted tasks, and organized expenses.
- The interface should support task feedback loop (e.g., flagging tasks and expenses that were wrongly created) with minimal user input.

3.4.2 Non-Functional Requirements

Accessibility and Usability:

- The platform should be usable by business users with no technical background, with minimal learning curve.
- The UI must provide clean visualizations of categorized messages, extracted expenses, and grouped projections.
- Support for Indian languages and voice-based commands is recommended to enhance inclusivity.

Accuracy and Reliability:

- OCR and LLM models must be optimized for high accuracy, especially in multilingual and noisy chat scenarios.
- The projection similarity model must achieve high precision to ensure critical updates are never mistakenly grouped.

Scalability:

- The system should be able to handle WhatsApp chats from multiple groups and users concurrently.
- Cloud infrastructure (e.g., GCS) should be used to ensure horizontal scaling for new accounts and increasing data load.

Latency and Performance:

- Message parsing and classification should occur in near real-time (<2 seconds per message).
- Expense extraction from images must respond within 3–5 seconds under normal internet conditions.

Security and Privacy:

- All user data—including chat logs and extracted financial information—must be stored securely using encryption.
- Access control mechanisms must be enforced to prevent unauthorized viewing or tampering of sensitive business information.
- Data retention policies should be configurable to comply with enterprise policies and privacy regulations.

4. Proposed System

The proposed system is a multimodal AI framework that automates the extraction and classification of structured information from unstructured WhatsApp chats. It consists of two main operational flows: one for expense tracking and another for task creation. Both flows process user-generated messages from raw input to structured output, enabling seamless integration with financial records and task management tools.

4.1 Solution Overview

The system architecture is built around two core flows — the Expense Parsing Flow and the Task Creation Flow — each responsible for handling a distinct set of intents within user conversations. These flows are supported by a unified Projection Merging Module and a expense classification model, which employs a fine-tuned BERT model to intelligently reconcile overlapping data streams and ensure coherent representation in structured formats and detect expenses from non-expenses.

4.1.1 Expense Tracker Flow

The expense tracker flow is initiated when WhatsApp chats are ingested through the platform interface. The system sequentially processes these inputs as follows:

- Images/PDF's are first passed through a captioning layer to identify potentially relevant expense entries.
- A classification module then categorizes each caption into an expense, not expense or maybe expense.
- The extracted outputs are formatted through a OCR and GPT layer into structured representations such as tables, which can be visualized or stored.
- Users can then view the details of their expenses in a neat tabular format via the UI which can be exported.

This flow ensures that expenses mentioned in natural language chat are automatically converted into structured formats suitable for tracking and analysis.

4.1.2 Task Creation Flow

The task creation flow identifies and structures messages that indicate action items or commitments. The flow begins by creating conversations from the stream of messages that are sent on a group. Projections (tasks) are created from the conversations and If identified as task-oriented, the message is passed into the task classification pipeline, where its context and urgency are assessed. Subsequently, the message is transformed into a task projection output — which includes elements like task title, priority, and due which can be viewed in the UI tasks tab.

The BERT model plays a critical role in ensuring no duplicate tasks are created when new projections are created as the conversation updates iteratively. It ensures less cluttering of data on the frontend and makes the creation of tasks seamless with the normal flow of conversations.

4.2 Functional Specifications

4.2.1 Process flow diagram

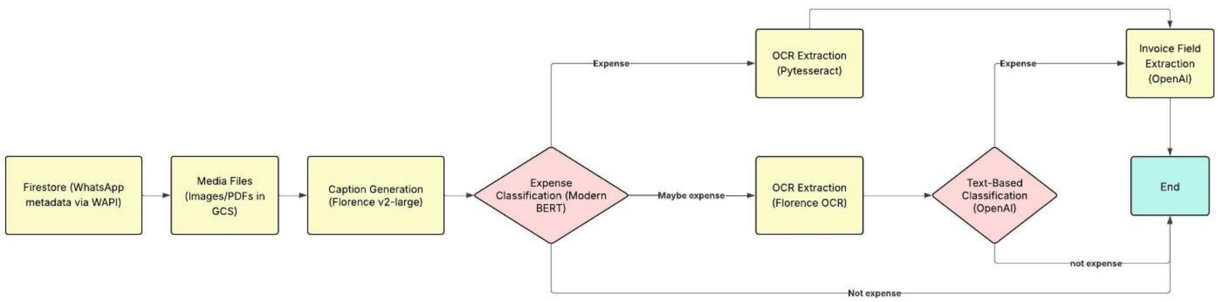

Figure 1: Workflow illustrating the automated extraction and classification of expense-related information from WhatsApp messages, followed by structured output generation using a fine-tuned BERT model.

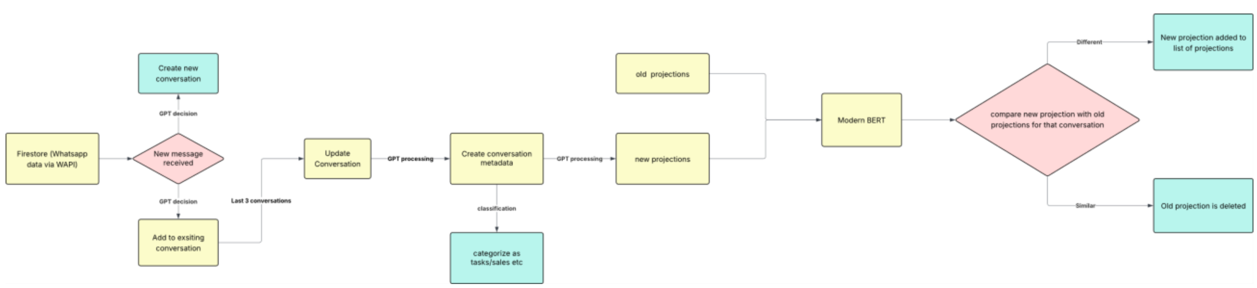

Figure 2: Workflow depicting the identification and structuring of task-related messages from WhatsApp chats, culminating in a unified output through a fine-tuned BERT-based projection mechanism.

4. 3 Feasibility Study

The feasibility study evaluates the technical, operational, and economic viability of the proposed multimodal AI system for structuring WhatsApp chats into actionable outputs. The system's components — including message classification, task and expense projection, and the BERT-based output unification — are analyzed for their robustness, accessibility, and integration potential. The study confirms that the architecture is scalable, practical for deployment, and adaptable to diverse user needs.

- Message Classification and Projection (Chetto.ai Integration): The Chetto.ai infrastructure supports seamless ingestion and classification of WhatsApp messages using pre-trained and fine-tuned models. These models are capable of identifying a range of intents including expense records, task items, and ambiguous instructions. The modular design allows continuous updates and retraining to improve classification accuracy. The classification system is lightweight and can be deployed both in cloud-based and edge environments, ensuring operational feasibility for varying device and connectivity scenarios.
- BERT-Based Projection Module (Merging and Classification): The system employs two key BERT-based capabilities. First, a fine-tuned BERT classifier evaluates captions generated from WhatsApp images to determine whether the message likely represents an expense or not — serving as the initial filter in the parsing pipeline. Second, a BERT similarity model compares new projections with previously generated ones in the same conversation thread to identify and eliminate semantic duplicates. This dual-use of BERT ensures both accurate classification and clean, non-redundant tracking of expenses and tasks, even across multilingual or informal message formats. Both models are optimized for fast, resource-efficient inference in real-time conditions.
- User Accessibility and Feedback Loop: A feedback mechanism is integrated into the workflow, enabling users to validate, edit, or discard AI-generated results. This user input is continuously logged to improve model performance over time, particularly in ambiguous or edge cases. Designed with accessibility in mind, the interface supports multiple languages and device types, encouraging widespread use across regions and technical skill levels.

This feasibility analysis demonstrates that the proposed system is both technically and operationally viable for real-world deployment, especially in domains where structured information needs to be extracted from informal, conversational input sources.

4.4 Design Modeling and Test Cases

The design of the system adopts a modular and extensible architecture to ensure that each component — from message parsing to classification and projection — functions independently while contributing to the larger workflow. This design allows for iterative development, scalability, and easy integration of future enhancements, including additional intent categories or output formats.

4.4.1 Model Components and Specifications

- Florence Captioning: The system utilizes Microsoft's Florence Captioning model as the first step in processing any image-based message shared via WhatsApp. This model generates a natural language caption describing the image, which serves as a semantic summary — for instance, recognizing a bill, invoice, or transaction slip. This caption becomes the foundation for downstream classification and helps handle informal or cluttered receipts where text is not easily extracted.

- BERT-Based Expense Classification fine-tuned BERT model consumes the image caption produced by Florence and determines whether the content likely represents an expense. This classification stage acts as a filter, ensuring that only financially relevant images move forward. If the caption does not imply an expense, the image is discarded early in the pipeline, reducing computational overhead for non-essential processing.
- Pytesseract OCR Engine: If the image is classified as an expense, it is passed through Pytesseract OCR, the primary text extraction engine. Pytesseract OCR is optimized for structured document recognition and extracts key textual elements such as amounts, vendor names, dates, and payment modes from the image.
- Florence OCR (Fallback): When Pytesseract OCR encounters noisy, low-contrast, or handwritten text and fails to deliver usable results, the Florence OCR model is triggered as a fallback. This ensures robustness by capturing difficult-to-read characters or low-quality media that standard OCR engines might skip or misread.
- GPT-Based Fallback Classification: In scenarios where the BERT classification or OCR results are ambiguous or contradictory, a GPT model is used to semantically infer whether the image content still qualifies as an expense. GPT combines both the caption and OCR output to make this determination, especially in edge cases where structured classifiers might fail due to informal formatting or vague wording.
- GPT-Based Structured Response Generator: Once the message is confirmed to represent an expense, GPT is employed a second time — now to generate a structured response. This includes extracting and formatting information such as amount, vendor, date, and category into a standardized JSON schema, making it ready for integration into downstream systems such as finance trackers or task managers.
- Modern BERT for Task Similarity Classification: To prevent redundant entries in ongoing WhatsApp conversations, a fine-tuned Modern BERT model is employed to assess semantic similarity between newly generated projections and existing ones within the same thread. This model compares contextual meaning rather than surface-level keywords, enabling it to recognize paraphrased or repeated intents. If a new projection is determined to be similar to a previous one, the system intelligently discards the old projection to maintain relevance and avoid duplication in downstream task or sales tracking flows.

4.4.2 Test Case Design and Results

Florence Captioning was evaluated on a diverse dataset of receipt and bill images to ensure the generated captions were semantically accurate and contextually useful. Test cases included images with poor lighting, partial obstructions, and handwritten text. BERT classification based on these captions was tested for its accuracy in flagging messages as expenses, maybe expenses or not, with precision and recall scores exceeding 97% in controlled evaluations.

- Captioning and Pre-OCR Classification: Florence Captioning was evaluated on a diverse dataset of receipt and bill images to ensure the generated captions were semantically accurate and contextually useful. Test cases included images with poor lighting, partial obstructions, and handwritten text. BERT classification based on these captions was tested for its binary accuracy in flagging messages as expenses or not, with precision and recall scores exceeding 97% in controlled evaluations.
- OCR Pipeline Robustness: Pytesseract OCR and Florence OCR were tested using varied WhatsApp-shared media, including scans, screenshots, and mobile photos. Pytesseract OCR served as the primary extraction engine, while Florence OCR was triggered in cases of OCR confidence below a defined threshold. Test cases validated correct extraction of amounts, vendor names, and bill dates. Accuracy was benchmarked against ground-truth text transcriptions, with fallback Florence OCR improving overall extraction success rate by 3% in degraded-image scenarios. Accuracy of Pytesseract OCR increased from 60% to 97% with the help of data preprocessing techniques.
- GPT-Based Structured Response Generation: GPT's ability to extract structured JSON outputs was validated using a test suite of 500 real-world messages. Evaluation focused on correct amount, vendor name etc being extracted. Human evaluators scored outputs on structure, completeness, and correctness, with GPT achieving a 98% average success rate in generating usable and contextually accurate records.

4.5 Estimation

4.5.1 Hardware Requirements

The system is built for cloud-native deployment using Google Cloud Platform (GCP), optimized for high-throughput WhatsApp message processing and model inference.

- Servers: Inference tasks for models such as BERT classifiers, are executed on GCP Compute Engine instances, optionally configured with CPUs to ensure low-latency performance during peak usage. These models are packaged and stored in Google Cloud Storage (GCS) buckets for flexible access and version control.
- User Devices: End users interact with the system via WhatsApp on standard mobile devices. No computation occurs on-device, which ensures accessibility even in low-resource or rural settings.
- Storage: Incoming WhatsApp messages and metadata are stored in Google Firestore, enabling efficient document-based queries and seamless integration with downstream processing.
- Model training and fine-tuning — including BERT-based classifiers, task similarity modules, and lightweight vision models — are conducted using Google Colab's Pro-tier GPU

environments (e.g., NVIDIA T4, A100). This setup supports rapid prototyping and experimentation without the overhead of provisioning dedicated cloud infrastructure.

4.5.2 Software Requirements

The system leverages a combination of Google Cloud services, open-source AI frameworks, and third-party APIs to manage the full pipeline—from message ingestion to structured data output.

- Frameworks: The system is built on PyTorch for serving BERT-based classifiers and handling GPT-related inference logic. Pytesseract OCR is integrated directly as a Python library, allowing local text extraction from user-submitted images without external calls. This reduces latency and improves control over OCR processing workflows.
- Third-Party AI Access via API:
  - Florence Captioning and Florence OCR are accessed through Replicated-hosted API endpoints to handle image understanding and fallback text extraction.
  - GPT models are invoked via the OpenAI API, enabling semantic validation and structured output generation without hosting large language models internally.
- APIs:
  - WhatsApp Business API is used to receive and manage incoming user messages (text, image, or mixed media).
  - Internal APIs, built using FastAPI, handle orchestration of captioning, classification, projection similarity checks, and structured response delivery.
- Backend Storage:
  - Google Firestore (GCP) stores WhatsApp message metadata, classification labels, and conversation threads.
  - Google Cloud Storage (GCS) is used for storing image files and media assets submitted by users for further processing.

4.5.3 Data Requirements

The system leverages multimodal datasets for training, validation, and inference tuning. These datasets ensure accurate classification of WhatsApp messages and reliable projection management across informal, multilingual, and image-based communication.

Image Data: A collection of WhatsApp-shared images including receipts, utility bills, handwritten notes, and digital payment confirmations is used to evaluate and validate the Florence Captioning and OCR modules. This dataset also supports robustness testing across various image formats and qualities.

Text Data: Real and synthetic WhatsApp messages representing diverse expense and task-related conversations are used to fine-tune BERT classifiers and optimize GPT prompting. This dataset captures informal phrasing, regional language usage, and code-mixed inputs common in everyday business chats.

Projection Data: Previously generated projections — including structured outputs from expense parsing and task recognition — are stored to train the BERT similarity model. These are used to evaluate whether new projections are redundant or distinct, enabling accurate semantic deduplication across message threads.

4.5.4 Cost and Scheduling

The project accounts for costs related to cloud infrastructure, third-party APIs, and storage services, with a development schedule organized around modular milestones.

- GCP Compute Engine (CPU with optional GPU) for serving BERT, Pytesseract OCR, and GPT inference logic
- Google Cloud Storage (GCS) for storing user-submitted images and media files
- Google Firestore for storing WhatsApp message metadata, conversation states, and structured outputs
- OpenAI API usage for GPT-based reasoning and structured response generation
- Replicated API access for Florence Captioning and OCR modules
- WhatsApp Business API fees for managing inbound message streams
- Model training via Google Colab GPUs, reducing the need for paid training infrastructure

5. Conclusion and Future Work

The multimodal AI system developed in this project demonstrates an effective and scalable solution for parsing expense-related information from informal WhatsApp messages. By integrating image captioning, OCR, semantic classification, and task similarity detection, the system transforms unstructured conversations into structured, actionable data — minimizing manual tracking and improving the accuracy of financial and operational insights.

The solution leverages lightweight, cloud-based inference to ensure compatibility with low-bandwidth mobile environments, enabling accessibility across diverse user groups. It also introduces redundancy handling via semantic similarity matching, making projections cleaner and more relevant over time.

Future work will focus on strengthening system robustness, expanding linguistic reach, and improving automation across the following dimensions:

- Multilingual Model Expansion: Integrating more vernacular and code-mixed language models for better understanding of local expressions and dialects used in rural or multilingual WhatsApp messages.
- Advanced Projection Linking: Enhancing the Modern BERT similarity model to support temporal threading and event grouping, allowing projections to be linked into conversational chains (e.g., invoice sent → payment confirmed → task closed).
- Low-Resource Optimization: Exploring on-device lightweight inference or hybrid edge-cloud processing to reduce API dependency and operational costs, while maintaining responsiveness in constrained environments.
- Feedback-Driven Learning Loops: Incorporating active learning mechanisms where user feedback can help retrain classifiers and GPT templates to improve accuracy over time, especially for edge cases and new expense formats.

These enhancements will further improve system adaptability, ensure higher accuracy in complex real-world contexts, and support deployment at scale across sectors beyond finance — including sales automation, task tracking, and conversational CRM.

6. References

- J. Yuan et al., Florence: A New Foundation Model for Computer Vision, arXiv:2111.11432, 2021. [Online]. Available: https://arxiv.org/abs/2111.11432
- J. Devlin, M. Chang, K. Lee, and K. Toutanova, BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, in Proc. NAACL-HLT, 2019. [Online]. Available: https://arxiv.org/abs/1810.04805
- T. Wolf et al., Transformers: State-of-the-Art Natural Language Processing, in Proc. EMNLP: System Demonstrations, 2020. [Online]. Available: https://www.aclweb.org/anthology/2020.emnlp-demos.6
- OpenAI, GPT-4 Technical Report, Mar. 2023. [Online]. Available: https://openai.com/research/gpt-4
- PaddlePaddle Contributors, Pytesseract OCR: Multi-language OCR System Based on PaddlePaddle, GitHub, 2020. [Online]. Available: https://github.com/PaddlePaddle/Pytesseract OCR
- Meta AI, LLaMA 2: Open Foundation and Fine-Tuned Chat Models, Meta AI, Jul. 2023. [Online]. Available: https://ai.meta.com/llama/
- Google Cloud, Firestore Documentation, 2024. [Online]. Available: https://cloud.google.com/firestore/docs
- Google Cloud, Cloud Storage Documentation, 2024. [Online]. Available: https://cloud.google.com/storage/docs
- Replicate, Florence Model Access via Replicated API, 2024. [Online]. Available: https://replicate.com/
- Twilio, WhatsApp Business API Overview, 2024. [Online]. Available: https://www.twilio.com/docs/whatsapp

7. Bibliography

- https://arxiv.org/abs/2111.11432
- https://arxiv.org/abs/1810.04805
- https://www.aclweb.org/anthology/2020.emnlp-demos.6
- https://openai.com/research/gpt-4
- https://github.com/PaddlePaddle/Pytesseract OCR
- https://ai.meta.com/llama/
- https://replicate.com/
- https://www.twilio.com/docs/whatsapp
- https://cloud.google.com/firestore/docs
- https://cloud.google.com/storage/docs
- https://cloud.google.com/compute/docs
- https://pytorch.org/
- https://fastapi.tiangolo.com/
- https://colab.research.google.com/
- https://developers.facebook.com/docs/whatsapp/overview
- https://platform.openai.com/docs
- https://huggingface.co/transformers/
- https://firebase.google.com/docs

8. Appendix

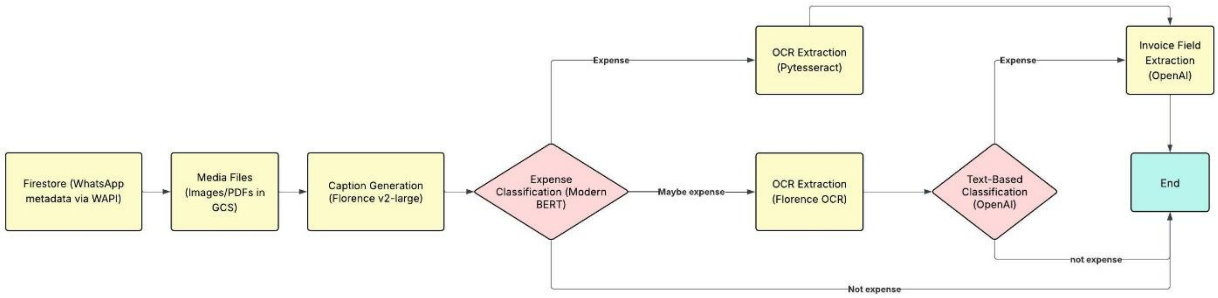

Figure 3: Workflow illustrating the automated extraction and classification of expense-related information from WhatsApp messages, followed by structured output generation using a fine-tuned BERT model.

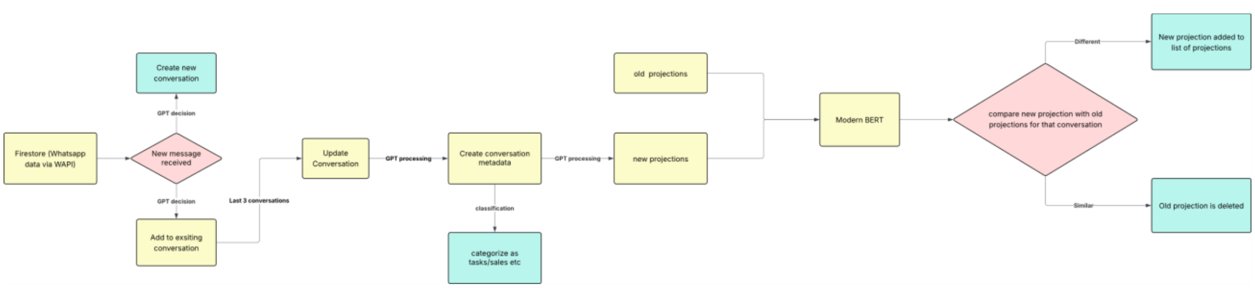

Figure 4: Workflow depicting the identification and structuring of task-related messages from WhatsApp chats, culminating in a unified output through a fine-tuned BERT-based projection mechanism.

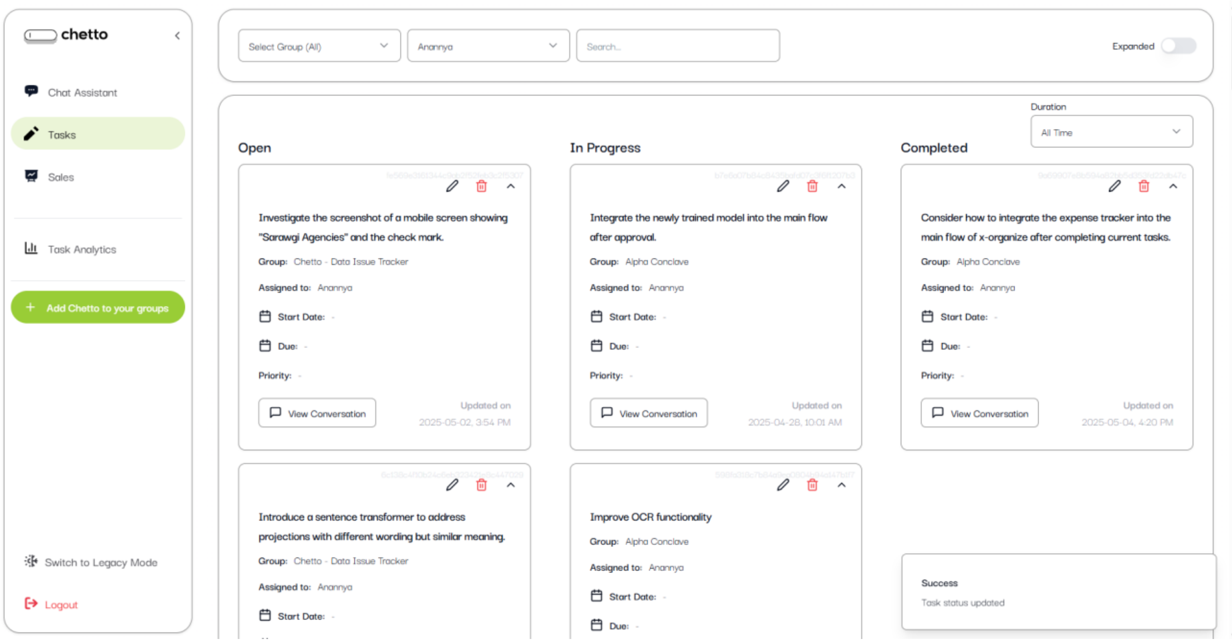

Figure 5: This image shows the users view of their tasks created based on the Whatsapp conversation they had on their respective group.

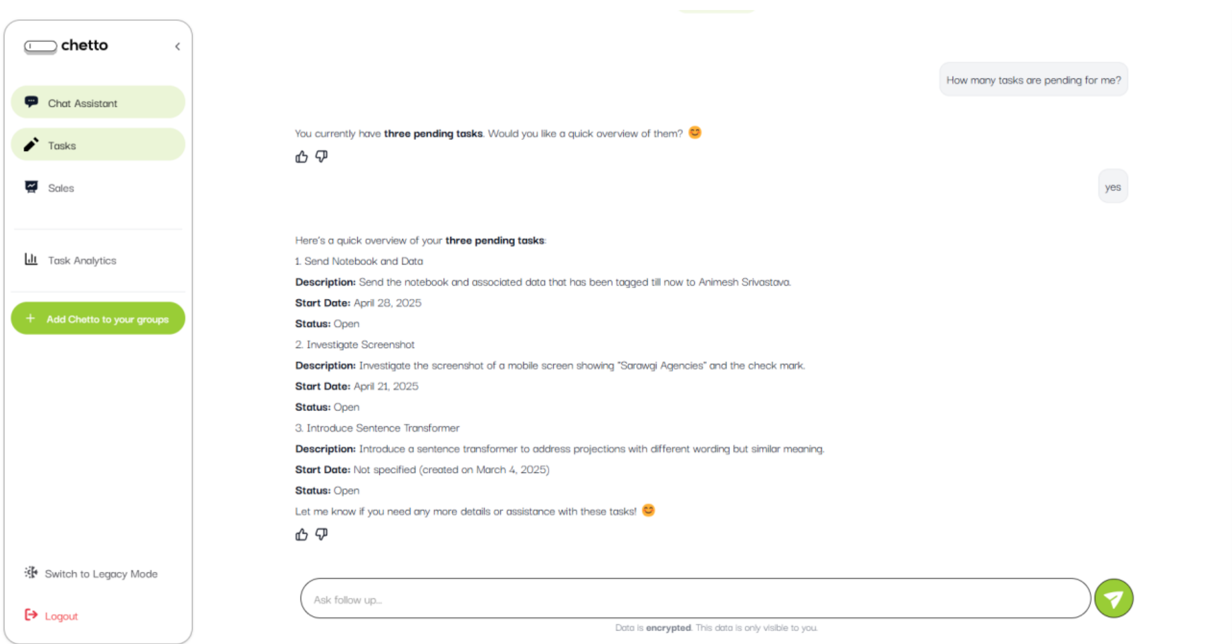

Figure 6: This images shows the chat assistant feature to be able to query your chats based on groups.

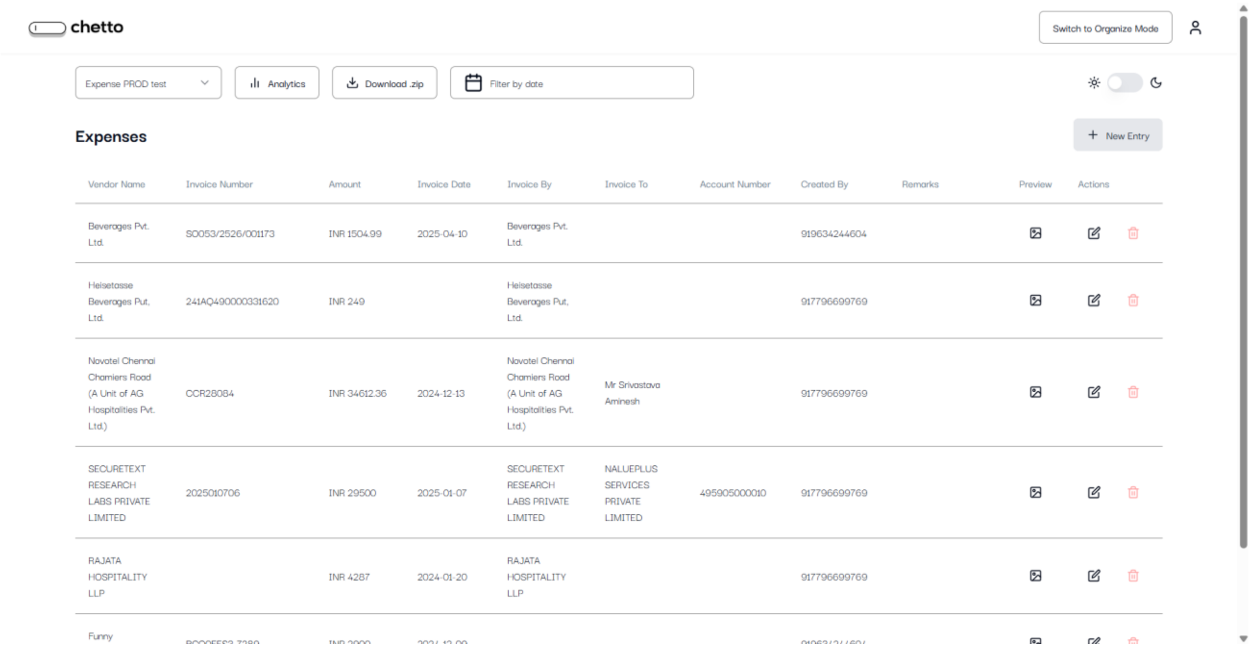

Figure 7: This image shows the UI for the expense tracker.